

DEAKIN UNIVERSITY

SOFTWARE ARCHITECTURE AND SCALABILITY FOR INTERNET-OF-THINGS

ONTRACK SUBMISSION

Module 2 Learning Evidence

Submitted By:

Jahan GARG

s226100103

2026/08/10 20:58

Tutor:

Nisha KUMARI

Task Outcomes		Supports
TLO1	Demonstrate engagement with the following unit learning outcomes (legacy)	ULO1 Demonstrate ULO2 Apply ULO3 Develop ULO4 Deploy

August 10, 2026



Week3_2026 Friday 24th July

meritnation.com/app/promotion/vapjgdeedvhlgs/https://a2b766826/vwehstm_ssmrc+gownebstm_nackun+buttwidqpxon=wo4736fh32e

Week3_2026 Friday 24th July

View only

SIT314/SIT729 - Week 3

How well you understand?

Class Discussion

1. Meaning of Curriculum

2. Curriculum

3. Curriculum

4. Curriculum

5. Curriculum

SIT314/SIT729 - Week 3 Seminar

24

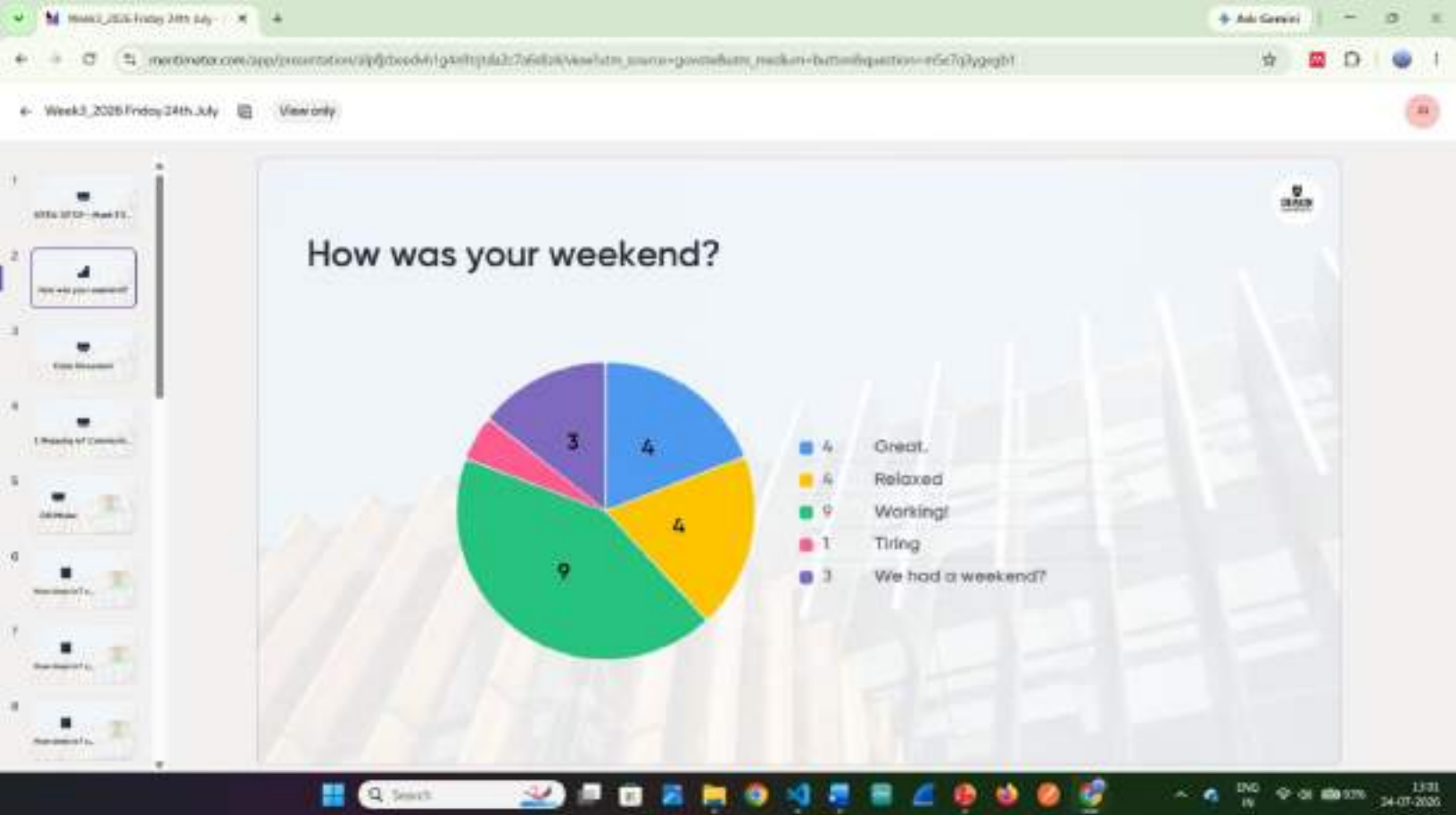
Windows Taskbar

Search

Taskbar Icons

System Tray

13:31 24-07-2026



Week3_2026 Friday 24th July

meritnation.com/app/promotion/vlpj0cedvhlgsxhplsl27x6ll2k/view.htm_source=google&utm_medium=button&question=ie67m6gnew

Week3_2026 Friday 24th July

View only

00:00:30:00 - Week 11

How well you understand?

Class Discussion

1. Mapping of Components

2. Low Power

3. Mapping of IoT

4. Low Power IoT

5. Low Power IoT

6. Low Power IoT

Class Discussion

1. Mapping IoT Communications to the OSI Model.
2. Low Power Communications.
3. Application Layer Communications.

13:31

24-07-2026

Week3_2026-Friday 24th July

meritnation.com/app/promotion/vip/00edvhlgs/htpda276682k/view.htm_source=google&utm_medium=button&utm_campaign=TheBestof44

Week3_2026-Friday 24th July

View only

100% 10/10 - Week 11

How well you prepared?

Class Discussion

1 Mapping IoT Communications

2 Overview

3 Introduction to IoT

4 Overview of IoT

5 Introduction to IoT

1. Mapping IoT Communications to the OSI Model

13:31

24-07-2026

Week3_2026-Friday 24th July

View only

OSI Model

7

UPPER LAYERS

Application Layer

✓ Message format, Human-Machine interfaces

6

Presentation Layer

✓ Coding into 1s and 0s; encryption, compression

5

Session Layer

✓ Authentication, permissions, session restoration

4

TRANSPORT MIDDLEWARE

Transport Layer

✓ End-to-end error control

3

Network Layer

✓ Network addressing, routing or switching

2

Data Link Layer

✓ Error detection, flow control on physical link

1

Physical Layer

✓ Bit stream, physical medium, method of representing bits

OSI Model

7

UPPER LAYERS

Application Layer

✓ Message format, Human-Machine interfaces

6

Presentation Layer

✓ Coding into 1s and 0s; encryption, compression

5

Session Layer

✓ Authentication, permissions, session restoration

4

TRANSPORT MIDDLEWARE

Transport Layer

✓ End-to-end error control

3

Network Layer

✓ Network addressing, routing or switching

2

Data Link Layer

✓ Error detection, flow control on physical link

1

Physical Layer

✓ Bit stream, physical medium, method of representing bits

13:31

24-07-2026



Application Layer	7	Application Layer	Message format, Internet Message
	6	Presentation Layer	Converting into bits and file encryption/compression
	5	Session Layer	Authentication, permissions, session termination
	4	Transport Layer	End-to-end error control
	3	Network Layer	Network addressing, routing of datagrams
	2	Data Link Layer	Wireless detection, flow control on physical link
	1	Physical Layer	Bit stream, physical medium, method of representing bit



	7	Application Layer • Message format, higher-level protocols
	6	Presentation Layer • Converting into bits and file encryption/compression
	5	Session Layer • Authentication, permissions, session termination
	4	Transport Layer • End-to-end error control
	3	Network Layer • Network addressing, routing, and switching
	2	Data Link Layer • Wire interfaces, flow control on physical link
	1	Physical Layer • Bit stream, physical medium, method of representing bit

Week3_2026 Friday 24th July

View only

How does IoT communications map to the Session/Presentation/Application Layer of the OSI Model?

connection management

data formatting

packets

https

http

coap

mqtt

encryption

Application Layer

Presentation Layer

Session Layer

Transport Layer

Network Layer

Data Link Layer

Physical Layer

13:31

24-07-2026

Week3_2026 Friday 24th July

meritnation.com/app/promotion/vip/gdcodevlgxvhtpda27x682x/view.htm_source=google&utm_medium=button&question=367q25jpy

Week3_2026 Friday 24th July

View only

2

Play with your segment!

3

Class Discussion

4

Mapping of concepts

5

Q&A

6

Week-3 Quiz 1

7

Week-3 Quiz 2

8

Week-3 Quiz 3

9

2 Day Power Communication

2. Low Power Communications

24

Search

24-07-2026 13:31

Week3_2026 Friday 24th July

Week3_2026 Friday 24th July

View only

1

2

3

4

5

6

7

8

9

10

Why do we need low power communications for IoT?

A word cloud titled "Why do we need low power communications for IoT?". The words are arranged in a circular pattern around the center. The most prominent words are "battery life", "efficiency", "longevity", and "long power". Other words include "energy efficiency", "low power", "system efficiency", "to extend battery life", "support remote devices", "more energy efficiency", "longer lifespan", "avoid obstruction", "save money", "extends device battery li", "ease of maintenance", "flexible", "emergency relays", "riskless primary power", "high battery life", "cost saving", "emergency systems", "high power consumption", "extended battery life", "long life", "reduced maintenance cost", "high power loss", "reduce energy consumption", "independence from grid", "induce power consumption", "change due to loss energy", "low cost", "reliability", "devices die quickly", "poor battery", "energy efficiency", "total stress", "independence from grid", "induce power consumption", "change due to loss energy".

13:31

24-07-2026

Week3_2026 Friday 24th July

meritnation.com/app/production/vqpfj0cedvhlgs4t8p16z7x6l8z4vse4tm_ssmc+gawtebum_nackun+buttnifquestion+jpgnrcjsul

Week3_2026 Friday 24th July

View only

1 Mapping of Characteristics

2 Overview

3 Introduction to LoWPAN

4 Characteristics of LoWPAN

5 Low Power and Low Cost

6 Energy Efficient

7 Quick IPv6

8 Short Range Communication

9 Wireless Communication

10 Extra Layers

11 Why are these features important?

12 What are the challenges?

What are the characteristics of LoWPAN technologies?

aggressive sleep modes

low power consumption

fast

secure connection

low power efficient

limited bandwidth

low speed

unreliable

header compression

limited coverage

see 802154-2003

energy efficient consumer

low bandwidth

low data rate

short range

low power

low cost

more secure

wireless

compressed speed

small packet

low bandwidth

low power and low cost

energy efficient

quick IPv6

IPv6 support

short range communication

wireless communication

extra layers

13:31

24-07-2026

Week3_2020 Friday 24th July

meritnation.com/app/promotion/vip/gdceodvhtgslhplsl27x6ll2k/vsewlsn_ssnw-gwvtebunz_nackun-burmwifquestion=3ktp/hly84

Week3_2020 Friday 24th July

View only

0

Week 3_2020 Friday 24th July

7

Week 3_2020 Friday 24th July

8

Week 3_2020 Friday 24th July

9

Week 3_2020 Friday 24th July

10

Week 3_2020 Friday 24th July

11

Week 3_2020 Friday 24th July

12

Week 3_2020 Friday 24th July

13

Week 3_2020 Friday 24th July

3. Application Layer Communications

13:31

24-07-2020

Week3_2026 Friday 24th July

meritnode.com/app/promotion/vlpj0deedvhlgsfhtp1a27x6l2k/vse/str_ssrn=gwstebum_nackun=butmefixation=hw1qg0f0xw

Week3_2026 Friday 24th July

View only

7

Week3_2026 Friday 24th July

8

Week3_2026 Friday 24th July

9

Week3_2026 Friday 24th July

10

Week3_2026 Friday 24th July

11

Week3_2026 Friday 24th July

12

Week3_2026 Friday 24th July

13

Week3_2026 Friday 24th July

14

Week3_2026 Friday 24th July

What are the characteristics of RESTful communication?

statelessness

stateless

idempotent

lightweight

uri

uniform

http methods

client-server

lightweight using json

metadata in payload

client server architecture

slow transmission

get post put patch delete

client and server

req res

predictable

coherence

http requests

identity

https interactions

resource-based

uniform interface

13:31

24-07-2026



What do we need from an application layer communications protocol to build a scalable IoT application?

A word cloud of cloud computing characteristics. The words are arranged in a circular pattern, with some words being larger and more prominent than others. The words include: security, reliability, manages heavy data, scalability, pub sub model, interoperability, secure, low latency, fault tolerance, efficient messaging, specific standards, lightweight, portable, multi-tenant, elastic, distributed, and many others. The words are in various colors and sizes, creating a dynamic and visually appealing composition.

SIT314/SIT729 – Week 3 Group Activity

The Impact of IoT on our lives

Overview

This activity aims to plan the communications and connectivity for a distinct application.

Resource mining is a high-cost, high-reward industry, which rapidly adopts new technologies to manage those costs. IoT has had a substantial impact on automating mining. Dassault Systèmes has produced the following image, which summarises the use of IoT technologies in mining for their GEOVIA modelling software. For us, this provides a nice case study of the use of IoT in an environment that needs to be resilient and scalable.



This report analyses the Vehicle Asset Health application identified in the Dassault Systèmes / GEOVIA mining IoT case study (Dassault Systèmes/GEOVIA, n.d.), which highlights the use of connected sensing to "track operator productivity and vehicle asset health" across an open-pit and underground mining operation. Mining haul trucks, excavators and load-haul-dump (LHD) vehicles operate in harsh, remote, and safety-critical environments, making this

an instructive case study for scalable and resilient IoT system design consistent with the principles covered in Modules 1 and 2 of this unit (functional requirements, quality attributes, constraints, N-tier edge-fog-cloud architectures, and application-layer communication protocols) (Kua, Lecture 1.3, SIT314/SIT729).

This is an individually authored submission. The scenario and initial framing were discussed collaboratively with classmates Melissa Woo and Madhur Gupta during the Week 3 seminar; however, all analysis, architectural decisions, diagrams, and written content presented below are my own independent work.

Tasks

For the application 'Vehicle Asset Health' identified in the image.

1. What functionality is needed for this application? What should be sensed, what processing is needed and what actuation is possible?

1. Application Functionality: Sensing, Processing and Actuation

Following the functional requirements framework introduced in Module 1.3 (functional requirements, quality attribute requirements, and constraints), the Vehicle Asset Health application is decomposed below into what must be sensed, what processing is required at the edge and in the cloud, and what actuation the system can trigger in response.

1.1 Sensing requirements

Mining haul trucks and heavy vehicles are already instrumented with an on-board Electronic Control Unit (ECU) network conforming to SAE J1939, the recommended-practice serial control and communications vehicle bus used across on- and off-highway heavy equipment including construction and mining vehicles (SAE International, n.d.; Wikipedia, n.d.). Rather than retrofitting bespoke sensors, the most realistic and scalable design taps directly into this existing CAN/J1939 bus via a telematics gateway, consistent with real-world fleet telematics practice (Shukla, 2025).

- Engine coolant temperature and oil pressure (via J1939 Parameter Group Numbers, PGNs)
- Engine RPM, throttle position and engine load
- Battery/alternator voltage (control module voltage)
- Tyre pressure and temperature (TPMS — critical for extremely expensive mining tyres)
- GPS location and inertial (IMU) data for position, tilt and vibration monitoring
- Active Diagnostic Trouble Codes (DTCs) broadcast natively by the ECU as J1939 Diagnostic Messages (DM1/DM2) (SAE International, J1939/73, n.d.)
- Fuel level and fuel consumption rate
- Hydraulic system pressure (for excavators / LHD loaders)

1.2 Processing requirements

Processing is split across the edge gateway and the cloud, mirroring the N-tier edge-fog-cloud architecture discussed in Module 1.3 (OpenFog Consortium reference architecture). This split is essential given that mining sites frequently experience unreliable or intermittent backhaul

connectivity, and continuous streaming of raw high-frequency CAN data to the cloud would be both bandwidth-prohibitive and unnecessary.

Tier	Processing performed
Edge (on-vehicle telematics gateway)	Decode raw J1939 PGNs; apply local threshold checks (e.g., coolant temp > 105°C); timestamp and buffer data during connectivity loss; compress/aggregate before transmission; trigger immediate local safety actuation without waiting for cloud round-trip.
Cloud (fleet health platform)	Time-series storage of historical sensor streams; trend analysis and predictive maintenance modelling (e.g., detecting gradual oil-pressure decline preceding failure); fleet-wide health scoring; correlation across vehicles and against maintenance records; generation of consolidated dashboards and reports.

1.3 Actuation requirements

- In-cab visual/audible alert to the operator (immediate, edge-triggered)
- Automatic engine de-rating (power reduction) if a critical fault is confirmed, to prevent catastrophic failure
- Emergency engine shutdown command for the most severe safety-critical faults
- Automatic work-order / maintenance-ticket generation in the fleet management system
- SMS/email/push notification to the fleet manager and maintenance engineer
- Scheduling recommendation for the vehicle's next scheduled maintenance window

2. What web services would you need to structure to implement this?

2. Web Services Required

To support the functionality above, the cloud platform must expose a structured set of RESTful web services. These services correspond to the common IoT middleware functional requirements identified in Module 1.3 — automated resource discovery, resource management, data management and event management (Kua, Lecture 1.3, SIT314/SIT729).

Service	Purpose
Device Registration Service	Registers a new vehicle/telematics gateway; assigns a unique vehicleID and issues device credentials (supports automated resource discovery).
Data Ingestion Service	Receives batched/streamed sensor readings from the edge gateway and persists them to the time-series store.
Health & Diagnostics Service	Computes and exposes current health score, active DTCs, and predictive maintenance flags per vehicle.

Alert & Notification Service	Manages alert lifecycle (raise, acknowledge, escalate, resolve) and dispatches notifications to relevant stakeholders.
Maintenance Scheduling Service	Generates and tracks work orders; integrates with the mine's Computerised Maintenance Management System (CMMS).
Authentication & Access Control Service	Issues and validates tokens (OAuth2/JWT) for operators, fleet managers, engineers and OEM analysts, enforcing role-based access.
Dashboard / Reporting API	Serves aggregated fleet-wide and per-vehicle data to the web/mobile dashboard for visualisation.
Command Service	Securely relays actuation commands (de-rate, shutdown) from authorised personnel back down to a specific vehicle.

3. What hierarchical MQTT topics would you use?

3. Hierarchical MQTT Topics

As covered in Section 2.4 (MQTT), topics are hierarchical strings separated by '/' and clients may subscribe using single-level (+) or multi-level (#) wildcards for message filtering (Kua, Lecture 8.2.1, SIT314/SIT729). Because a single mine operates multiple pits and fleets, and a corporate operator may run multiple mine sites, the topic hierarchy is rooted at the site level rather than the vehicle level, to allow filtering at every level of the organisation — consistent with the cooperative broker-sharing convention discussed in the unit (prefixing topics with a unique identifier to avoid 'mess' on shared brokers) (Kua, Lecture 8.2.1, SIT314/SIT729).

Topic	Purpose
mine/{siteID}/vehicle/{vehicleID}/sensor/engine/temperature	Engine coolant temperature reading
mine/{siteID}/vehicle/{vehicleID}/sensor/engine/rpm	Engine RPM
mine/{siteID}/vehicle/{vehicleID}/sensor/battery/voltage	Control module/battery voltage
mine/{siteID}/vehicle/{vehicleID}/sensor/tyre/pressure	TPMS reading
mine/{siteID}/vehicle/{vehicleID}/sensor/oil/pressure	Engine oil pressure
mine/{siteID}/vehicle/{vehicleID}/location	GPS coordinates
mine/{siteID}/vehicle/{vehicleID}/health/score	Computed composite health score
mine/{siteID}/vehicle/{vehicleID}/health/status	Overall status: nominal/degraded/critical
mine/{siteID}/vehicle/{vehicleID}/alert/warning	Non-critical alert requiring monitoring

mine/{siteID}/vehicle/{vehicleID}/alert/critical	Safety-critical alert requiring immediate action
mine/{siteID}/vehicle/{vehicleID}/command/derate	Cloud-to-vehicle command: reduce engine power
mine/{siteID}/vehicle/{vehicleID}/command/shutdown	Cloud-to-vehicle command: emergency shutdown
mine/{siteID}/vehicle/{vehicleID}/command/ack	Vehicle acknowledgement of a received command

4. Create a high-level block diagram of the scenario; include all the people, places and things that are involved in the scenario.

4. High-Level Block Diagram and Wireless Technology Selection

The block diagram below identifies all people, places and things involved in the Vehicle Asset Health scenario, structured across the Mine Site, Network and Cloud tiers consistent with the N-tier deployment architecture (Edge → Fog → Cloud) discussed in Module 1.3 (Kua, Lecture 1.3, SIT314/SIT729).



Figure 1. High-level block diagram of people, places and things in the Vehicle Asset Health scenario.

4.1 Wireless technology decision

Section 2.1 of this unit establishes that long-range connectivity (>1 km up to 100 km) is provided either by licensed cellular technologies (3GPP GSM/WCDMA/LTE/NB-IoT) or by unlicensed LPWA technologies such as Sigfox and LoRa, while short-range technologies (Zigbee, 6LoWPAN, Bluetooth, Wi-Fi) are intended for local network access (Kua, Lecture 3.1, SIT314/SIT729). Applying this framework to a mining haul truck rules out short-range technologies entirely: haul trucks travel continuously across pit benches, ramps and haul roads spanning several kilometres, and a vehicle asset health system cannot depend on a fixed local hub.

Among the wide-area options, the choice is between LPWAN (LoRa/Sigfox/NB-IoT) and standard cellular (LTE/4G/5G or the LTE-M variant). LPWAN technologies are explicitly characterised in Section 2.3 as suited to "delay-tolerant" use cases exchanging "small amounts of data infrequently" and as unsuitable for reliability-sensitive vehicle applications (Kua, Lecture 3.3, SIT314/SIT729). A moving haul truck generating continuous multi-parameter telemetry, including safety-critical fault alerts that must reach the operator and fleet manager with low latency, does not fit this profile. NB-

IoT is also unsuitable for this scenario specifically because it does not natively support the cell-tower handover required for a continuously moving vehicle, and its throughput is optimised for small, infrequent payloads rather than multi-sensor streaming telemetry (iot.cards, n.d.; TopFlyTech, n.d.).

The recommended technology is therefore LTE-M (LTE for Machines / Cat-M1), a licensed cellular variant that retains full mobility and handover support (unlike NB-IoT), while still offering lower power consumption and lower module cost than full LTE/4G data plans, and considerably higher throughput and reliability than any LPWAN option (iot.cards, n.d.; TopFlyTech, n.d.). On very remote sites beyond cellular coverage (a realistic constraint for outback and underground Australian mines), the design includes a satellite backhaul fallback at the fog/edge gateway to preserve the resilience quality attribute emphasised throughout this module. Short-range Bluetooth or Wi-Fi is retained only for the local link between on-vehicle sensors and the telematics edge gateway itself (e.g., BLE TPMS sensors reporting to the in-cab unit), not for the wide-area backhaul.

5. Create a simple data flow diagram showing all the needed communication in this system.

5. Data Flow Diagram

The diagram below traces every communication step required in the system, from raw sensing through to stakeholder consumption, distinguishing data stores, processing nodes and external entities.

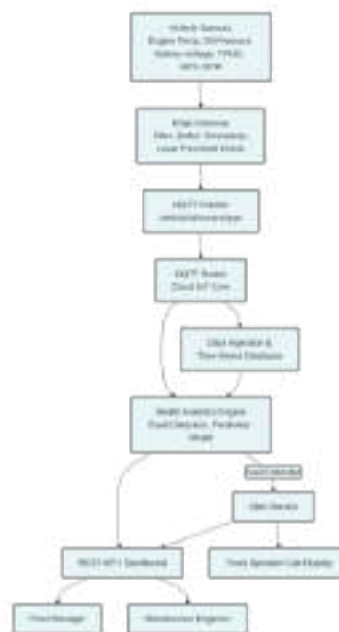


Figure 2. Data flow diagram for the Vehicle Asset Health application

6. Plan a web service-based approach to implementing the application-level communications for this application. What services would you need?

6. Approach A — Web Service (REST)-Based Application Communications

In a purely RESTful design, the telematics gateway acts as an HTTP client that periodically POSTs batched sensor readings to the Data Ingestion Service, and dashboards/apps poll GET endpoints for the latest state. This mirrors a request/response, synchronous style rather than the asynchronous pub/sub style of MQTT.

Endpoint	Method & Purpose
/vehicle/register	POST — register a new vehicle/gateway and obtain credentials
/vehicle/{id}/data	POST — submit a batch of sensor readings
/vehicle/{id}/status	GET — retrieve current status/health snapshot
/vehicle/{id}/history	GET — retrieve historical sensor data for a time range
/vehicle/{id}/health	GET — retrieve computed health score and predictive flags
/vehicle/{id}/alert	POST — raise an alert; GET — retrieve active alerts
/vehicle/{id}/command	POST — issue a de-rate/shutdown command (authorised users only)

Limitation: REST is inherently request/response and does not scale well for real-time, event-driven alerting across a fleet of hundreds of vehicles — the dashboard/operator app would need to continuously poll every vehicle's status endpoint, wasting bandwidth and increasing latency for safety-critical alerts, precisely the kind of 'synchronous I/O' challenge to scalability identified in Module 1.1 (Kua, Lecture 1.1, SIT314/SIT729).

7. Plan a hierarchical MQTT application-level communications protocol. Think about the identity of users and devices in the system and plan single-level or multi-level wildcards for message filtering.

7. Approach B — Hierarchical MQTT Application Communications

The MQTT approach (Section 3, above) instead uses the publish/subscribe model described in Section 2.4: the vehicle gateway publishes to its own topic branch, and a broker distributes messages only to interested subscribers, with no synchronous polling required (Kua, Lecture 8.2.1, SIT314/SIT729).

7.1 Identity design

- Each vehicle gateway authenticates to the broker with a unique client ID and publishes only under its own mine/{siteID}/vehicle/{vehicleID}/ branch (write scope restricted by broker ACL).

- Each human role (operator, fleet manager, maintenance engineer, OEM analyst) authenticates with role-specific credentials and is granted subscribe-only access to relevant branches.

7.2 Wildcard subscription design

Stakeholder	Subscription topic filter
Truck operator (single vehicle, all data)	mine/site01/vehicle/TRK-014/# (multi-level wildcard — all sensor, health, alert and command topics for their own vehicle only)
Fleet manager (all vehicles, alerts only)	mine/site01/vehicle/+/alert/# (single-level wildcard on vehicleID, multi-level on alert sub-topics — sees every vehicle's alerts fleet-wide)
Maintenance engineer (all vehicles, health only)	mine/site01/vehicle/+/health/# (fleet-wide health scores and status, ignoring raw sensor noise)
OEM reliability analyst (all sites, engine temperature trend only)	mine/+/vehicle/+/sensor/engine/temperature (cross-site analysis of a single parameter)

This wildcard design directly demonstrates the single-level (+) versus multi-level (#) distinction taught in Section 2.4: + substitutes exactly one topic level, such as any vehicleID or any siteID, while # subscribes to an entire branch and everything beneath it, regardless of depth (Kua, Lecture 8.2.1, SIT314/SIT729). The MQTT approach is more scalable than the REST approach because the broker, rather than each client, is responsible for filtering and fan-out, meaning new subscribers can attach to any level of the hierarchy without requiring any change to how vehicles publish data.

MQTT Topic Hierarchy — Visual Summary

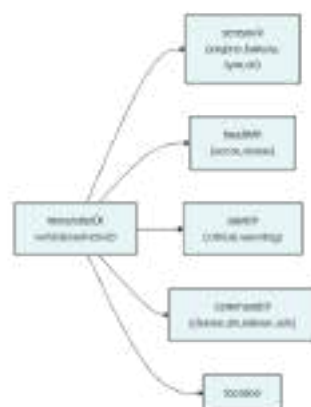


Figure 3. Hierarchical MQTT topic tree for the Vehicle Asset Health application

8. Comparison and Recommendation

Criterion	REST	MQTT
Communication style	Synchronous request/response	Asynchronous publish/subscribe
Real-time alerting	Poor — requires polling	Excellent — instant push to subscribers
Bandwidth efficiency	Higher overhead (HTTP headers)	Lightweight, low-bandwidth (Kua, Lecture 8.2.1, SIT314/SIT729)
Scalability to fleet size	Degrades as polling frequency multiplied by vehicle count grows	Broker fan-out scales independently of publisher count
Best fit in this design	External dashboards, reporting, historical queries, maintenance workflow (CRUD)	Real-time sensor telemetry, health status, alerts, and command/control

The recommended final architecture is therefore hybrid: MQTT for all time-sensitive, high-frequency, event-driven vehicle-to-cloud and cloud-to-vehicle traffic, with REST APIs layered on top of the cloud platform purely for dashboard queries, historical reporting, device registration and maintenance-workflow integration. This hybrid pattern reflects real-world commercial IoT platforms, such as AWS IoT Core and Azure IoT Hub, which expose MQTT ingestion alongside REST management APIs.

9. Reflection: Application Going Forward

This activity reinforced that technology selection in IoT architecture cannot be made in isolation from the physical constraints of the deployment environment. Mobility, remoteness, and safety-criticality, rather than convenience or default assumptions, should drive the choice between short-range and wide-area connectivity, and between LPWAN, NB-IoT, and LTE-M. Designing the MQTT topic hierarchy also demonstrated that identity and access boundaries, meaning which stakeholder needs which slice of data, must be planned before the topic structure is finalised, since retrofitting access control onto a poorly structured hierarchy is far harder than designing it in from the outset. I intend to apply this same discipline of matching connectivity technology and message-filtering design to real physical and organisational constraints when designing my Distinction project's IoT solution later in this unit.

References

Boser, B. E. MQTT lecture slides. UC Berkeley. Retrieved from <https://people.eecs.berkeley.edu/~boser/courses/49/lectures/L08%20MQTT.pdf>

Dassault Systèmes / GEOVIA. Internet of Things & Digital Connectivity in Mining [Image]. Retrieved from unit site, SIT314/SIT729, Week 3 Group Activity.

iot.cards. LTE-M vs NB-IoT: which to choose in 2026. Retrieved from <https://iot.cards/en/resources/comparisons/lte-m-vs-nb-iot>

Kua, J. Lecture 1.1 – What is Scalability? SIT314/SIT729, Deakin University, Trimester 2, 2026.



Kua, J. Lecture 1.3 – A Software Engineering Approach for Scalable IoT. SIT314/SIT729, Deakin University, Trimester 2, 2026.

Kua, J. Lecture 3.1 – IoT Connectivity Technologies. SIT314/SIT729, Deakin University, Trimester 2, 2026.

Kua, J. Lecture 3.2 – Low-Power Wireless Personal Area Networks. SIT314/SIT729, Deakin University, Trimester 2, 2026.

Kua, J. Lecture 3.3 – Low-Power Wide-Area Networks. SIT314/SIT729, Deakin University, Trimester 2, 2026.

Kua, J. Lecture 8.2.1 – Message Queuing Telemetry Transport. SIT314/SIT729, Deakin University, Trimester 2, 2026.

Kushalnagar, N., Montenegro, G., and Schumacher, C. (2007). IPv6 over Low-Power Wireless Personal Area Networks (6LoWPANs): Overview, Assumptions, Problem Statement, and Goals. RFC 4919, IETF.

Mekki, K., Bajic, E., Chaxel, F., and Meyer, F. (2018). A comparative study of LPWAN technologies for large-scale IoT deployment. ICT Express.

SAE International. J1939/73: Application Layer – Diagnostics Recommended Practice. Retrieved from https://saemobilus.sae.org/standards/j193973_200609-application-layer-diagnostics

SAE International. SAE J1939 Standards Collection. Retrieved from <https://www.sae.org/standards/development/ground-vehicle/sae-j1939-standards-collection-on-the-web>

Shukla, M. (2025). Remote Vehicle Health Monitoring System using ESP32, MCP2515 and OBD-II CAN Protocol [LinkedIn post]. Retrieved from https://www.linkedin.com/posts/mayank-shukla-24086112a_embeddedsystems-automotiveiot-obdii-activity-7357455202867662848-6Hu8

TopFlyTech. How to Choose Between LTE-M, NB-IoT, and 4G CAT-1. Retrieved from <https://www.topflytech.com/en/how-to-choose-lte-m-nb-iot-4g-cat-1-fleet-tracking/>

Wikipedia. SAE J1939. Retrieved from https://en.wikipedia.org/wiki/SAE_J1939

SIT314/SIT729 Week 3 Technical Task

Machine-to-Machine Communication with MQTT: A Rent-a-Drone Monitoring Case Study

Prepared by: Jahan Garg (Student ID: 226100103)
Unit: SIT314/SIT729 – Software Architecture and Scalability for Internet-of-Things
Task: Week 3 Technical Task – MQTT Publish/Subscribe Communication
Date: 10 August 2026, Week 3, Trimester 2, 2026

Overview

This report documents my completion of the Week 3 Technical Task, which introduces machine-to-machine communication for Internet-of-Things applications through the Message Queuing Telemetry Transport (MQTT) protocol. The task is framed around a fictional Rent-a-Drone company scenario, where an owner needs to monitor many drones that are simultaneously in flight or stationary, using a lightweight publish/subscribe messaging system rather than a request/response model. The exercise is split into two parts: first, operating MQTT directly from the command line against a public broker to observe topic hierarchy and wildcard filtering, and second, writing a Node.js publisher (client.js) and subscriber (server.js) pair to reproduce the same behaviour in code. Both parts were completed in full, and this report presents the process, the evidence collected, and my analysis of what each result demonstrates about MQTT as a scalable communication protocol for IoT.

Before beginning this task, I had prior hands-on experience with the Arduino Nano 33 IoT and Arduino UNO R3 microcontrollers from earlier project work, although I had not previously worked with the ESP32 board referenced in some of the unit's device platform material. This background gave me a reasonable starting familiarity with embedded device connectivity, but MQTT itself, and the concept of a shared broker mediating communication between many independent publishers and subscribers, was new to me and is the central learning outcome of this task.

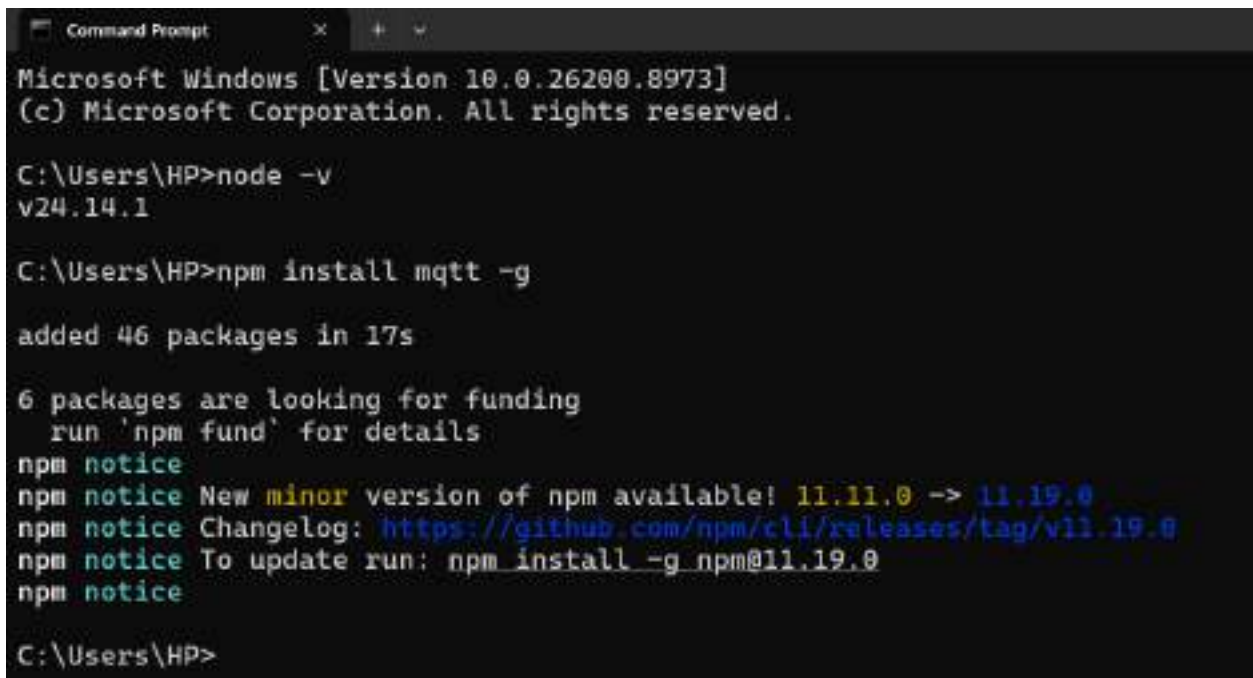
Part 1: MQTT from the Command Line

The first part of the task required setting up the MQTT command line tool and using it against a public broker to observe how topics and wildcard subscriptions govern message delivery. I used the free public broker hosted at `broker.hivemq.com`, which HiveMQ makes openly available for experimentation and testing purposes without requiring any account registration or credential setup [1]. Since the broker is shared globally by many users at once, I adopted a unique topic prefix based on my own name and student identifier (jahan226100103) to avoid any collision with other users' messages, following the convention discussed in the unit's MQTT lecture material regarding cooperative sharing of public brokers [2].

Environment Setup

Node.js was already installed on my machine from the Week 1 Technical Task, so I began by confirming the installed version and then installing the `mqtt` command line package globally using the

Node Package Manager (npm), which allows the mqtt command to be invoked directly from any terminal session rather than from within a specific project folder [3].



```
Microsoft Windows [Version 10.0.26200.8973]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP>node -v
v24.14.1

C:\Users\HP>npm install mqtt -g

added 46 packages in 17s

6 packages are looking for funding
  run 'npm fund' for details
npm notice
npm notice New minor version of npm available! 11.11.0 -> 11.19.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.19.0
npm notice To update run: npm install -g npm@11.19.0
npm notice

C:\Users\HP>
```

Screenshot 1 – Terminal output confirming the Node.js version (v24.14.1) and a successful global installation of the mqtt command line package (46 packages added).

This single, consolidated screenshot was sufficient to evidence that both prerequisites, Node.js itself and the mqtt CLI tool, were correctly installed before proceeding to the publish/subscribe experiments.

Basic Publish and Subscribe Test

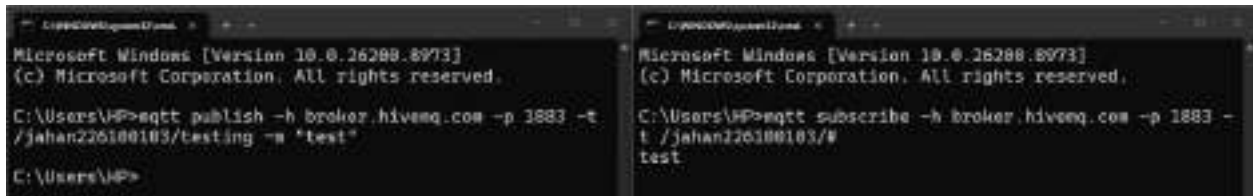
With the tooling confirmed, I opened a new terminal window and subscribed to my entire topic namespace using the multi-level wildcard operator, which subscribes to a topic and every level beneath it, regardless of depth [4].

```
mqtt subscribe -h broker.hivemq.com -p 1883 -t /jahan226100103/#
```

This terminal produced no visible output immediately after running, which is the expected behaviour for an MQTT subscriber: the process does not print anything until a message actually arrives on a matching topic, so it simply waits, connected and listening. I then opened a second terminal window and published a short test message to a subtopic of my namespace.

```
mqtt publish -h broker.hivemq.com -p 1883 -t /jahan226100103/testing -m "test"
```

Encountered issue: when I first attempted this publish command inside Windows PowerShell, the shell rejected it with a security error, stating that running scripts was disabled on the system, because npm installs its command line wrappers as PowerShell script files (.ps1), and PowerShell's default execution policy blocks unsigned scripts from running [5]. I resolved this by switching to the standard Windows Command Prompt (cmd.exe) instead, which invokes the equivalent .cmd wrapper and is not subject to the same script execution policy, allowing the command to run without needing to change any system security settings.



Screenshot 2 – Command Prompt showing the publish command executed successfully on the left, with the wildcard subscriber terminal on the right immediately displaying the received message 'test'.

The message appeared in the wildcard subscriber terminal exactly as expected, confirming that the broker had correctly routed the published message to every client subscribed to a matching topic branch.

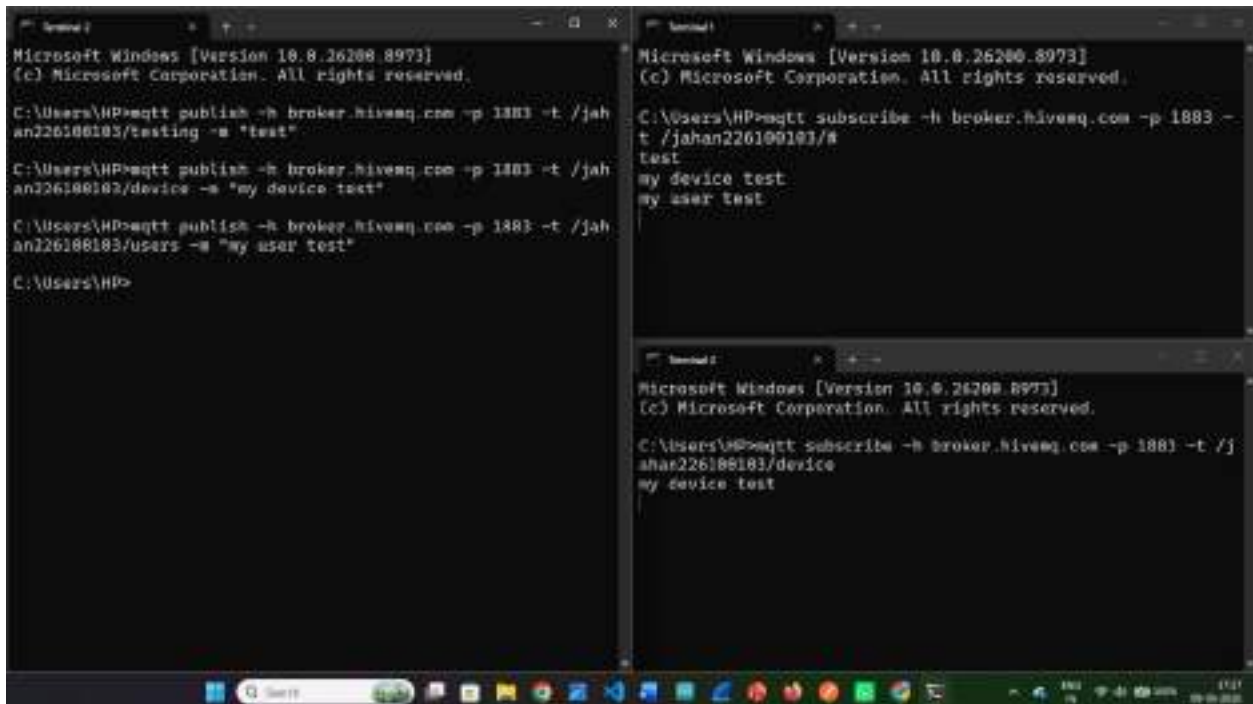
Subtopic Filtering and Wildcard Behaviour

The second half of Part 1 required demonstrating the difference between a wildcard subscription and a subscription to one specific subtopic. I opened a third terminal window and subscribed only to the /jahan226100103/device subtopic, without any wildcard.

```
mqtt subscribe -h broker.hivemq.com -p 1883 -t /jahan226100103/device
```

As with the wildcard subscriber, this terminal produced no output immediately, since it was simply waiting for a message published specifically to that exact topic string. From the publishing terminal, I then sent a message to that subtopic.

```
mqtt publish -h broker.hivemq.com -p 1883 -t /jahan226100103/device -m "my device test"
```



Screenshot 3 – Three terminals arranged together: the publishing terminal on one side, and both the wildcard subscriber and the specific-subtopic subscriber on the other, both displaying 'my device test' at the same moment.

This message appeared in both the wildcard subscriber (Terminal 1) and the specific subtopic subscriber (Terminal 3), since `/jahan226100103/device` matches the pattern of both `/jahan226100103/#` and `/jahan226100103/device` exactly. To isolate the filtering behaviour, I then published a message to a different subtopic that the specific subscriber was not listening to.

```
mqtt publish -h broker.hivemq.com -p 1883 -t /jahan226100103/users -m "my user test"
```

This message appeared only in the wildcard subscriber terminal and was correctly absent from the specific subtopic subscriber, since `/jahan226100103/users` does not match the exact topic string `/jahan226100103/device`. This single test pair is the clearest demonstration in the entire task of MQTT's hierarchical topic filtering: a client subscribed with the multi-level wildcard (`#`) receives every message published beneath the given prefix regardless of how many additional levels the topic contains, whereas a client subscribed to one exact topic string receives only messages published to that precise topic, and no others, even if they share the same parent branch [4].

Part 1 Result Summary

Message Published To	Received by Wildcard Subscriber (/jahan226100103/#)	Received by Specific Subscriber (/jahan226100103/device)
/jahan226100103/testing	Yes	N/A (not yet subscribed at this point)
/jahan226100103/device	Yes	Yes
/jahan226100103/users	Yes	No

This table summarises the observed delivery pattern across the three published messages and directly evidences why MQTT's hierarchical addressing model, described in the unit's Message Queuing Telemetry Transport lecture material as a means of organising related data streams under a common namespace, is considered scalable: an application such as the Rent-a-Drone monitoring dashboard can subscribe once with a wildcard to receive telemetry from every drone in a fleet, while a more specialised client, such as a single technician monitoring only one drone's battery status, can subscribe narrowly and avoid being flooded with irrelevant traffic from the rest of the fleet [2].

Part 2: Writing MQTT Publisher and Subscriber Programs in Node.js

The second part of the task moved from command line experimentation to writing actual application code, implementing the publisher (representing a drone transmitting data) and the subscriber (representing the owner's monitoring dashboard) as separate Node.js scripts that both connect to the same public broker programmatically using the `mqtt` npm package [3].

Project Setup

I created a dedicated project folder and initialised it as an npm project, then installed the `mqtt` package as a saved dependency so that it would be recorded in the project's `package.json` file



```
Microsoft Windows [Version 10.0.26200.8973]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP>mkdir week_3

C:\Users\HP>cd week_3

C:\Users\HP\week_3>npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See 'npm help init' for definitive documentation on these fields
and exactly what they do.

Use 'npm install <pkg>' afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (week_3)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
author:
license: (ISC)
type: (commonjs)
About to write to C:\Users\HP\week_3\package.json:
{
  "name": "week_3",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

Is this OK? (yes)
```

Screenshot 4 – Terminal output showing the npm init sequence completing and the mqtt package installing successfully, with the resulting package.json dependency entry visible.

Publisher Script (client.js)

The publisher script connects to the broker and, once the connection is acknowledged, publishes a single message to a fixed topic. In the MQTT protocol, this acknowledgement is the CONNACK message, which the mqtt.js client library surfaces to the application as an on('connect') event, following an asynchronous, event-driven pattern that is idiomatic to Node.js [3].

```
const mqtt = require('mqtt');
const client = mqtt.connect("mqtt://broker.hivemq.com:1883");

var topic = "/jahan226100103";
var message = "My message";

client.on('connect', () => {
  console.log('mqtt connected');
  client.publish(topic, message);
  console.log('published to Topic: ' + topic + " with Message: " + message);
});
```

Subscriber Script (server.js)

The subscriber script mirrors the publisher's connection logic but, instead of publishing on connect, it subscribes to the same topic and registers a separate listener for the on('message') event, which fires every time a message arrives on any topic the client has subscribed to.

```
const mqtt = require('mqtt');
const client = mqtt.connect("mqtt://broker.hivemq.com:1883");

var topic = "/jahan226100103";

client.on('connect', () => {
  client.subscribe(topic);
  console.log('mqtt connected');
});

client.on('message', (topic, message) => {
  console.log("Topic is: " + topic);
  console.log("Message is: " + message);
});
```

It is worth noting that the original task sheet's instructions refer to running these files with `node server.py` and `node client.py`. As both scripts are written in JavaScript rather than Python, I corrected this when running the files, using `node server.js` and `node client.js` instead, which is the correct invocation for a Node.js script.

Running and Verifying the Exchange

I ran the subscriber first, so that it would already be connected and listening before the publisher sent anything, and then ran the publisher in a second terminal within the same project folder.





Screenshot 5 – Two terminals shown together with the client.js and server.js code visible in the editor: the server.js terminal printing 'mqtt connected' followed by 'Topic is: /jahan226100103' and 'Message is: My message', and the client.js terminal printing its own 'mqtt connected' and 'published to Topic' confirmation.

The exchange completed exactly as expected: the subscriber correctly logged both the topic name and the message content it received, confirming that the two independently running Node.js processes had successfully communicated with each other purely through the intermediary broker, with no direct network connection between the two scripts themselves. This is the core architectural property of MQTT that differentiates it from a direct socket-based exchange, as used in the Week 1 and Week 2 Technical Tasks: neither client needs to know the other's network address, and either could disconnect and reconnect independently without the other needing to be reconfigured [2].

Cross-Verification with an Independent Publisher

To confirm that the subscriber was genuinely listening for any message on the matching topic, rather than specifically tied to my own client.js script, I kept server.js running and used the command line mqtt tool in a separate terminal to publish directly to the same topic.

```
mqtt publish -h broker.hivemq.com -p 1883 -t /jahan226100103 -m "my user test"
```

This step did not succeed on the first two attempts, which produced no visible message in the server.js console despite the publish command appearing to run without error. After some troubleshooting, I restarted server.js fresh, waited a few seconds for the connection and subscription to be fully acknowledged by the broker before publishing, and retyped the publish command rather than reusing it from terminal history. On this attempt, the message was received correctly.



Screenshot 6 – The server.js terminal on the left displaying 'mqtt connected', followed by 'Topic is: /jahan226100103' and 'Message is: my user test', with the independent publish command visible in the terminal on the right.

Observations and Troubleshooting

Two practical issues arose during this task that are worth documenting, since resolving them required understanding the underlying cause rather than simply retrying blindly, and both are genuinely relevant to the unit's broader themes of scalability and reliability in distributed systems.

The first issue, the PowerShell execution policy error, was a local environment configuration matter rather than an MQTT problem. PowerShell applies a default execution policy that restricts unsigned scripts from running, and because npm exposes globally installed command line tools as PowerShell wrapper scripts on Windows, this policy blocked the mqtt command specifically inside PowerShell sessions [5]. Rather than permanently altering the system's execution policy, which would weaken a security control for the sake of convenience, I judged it more appropriate to simply run the same command from the standard Command Prompt shell instead, which uses a different wrapper format not subject to the same restriction.

The second issue, the intermittent delay in the cross-verification step, is more directly relevant to this unit's content on scalability and connectivity reliability. Broker.hivemq.com is a globally shared, free public test broker used simultaneously by an unknown number of other students, developers, and automated test scripts around the world at any given time [1]. When a new client connects and subscribes, there is a brief window before the broker has fully processed the SUBSCRIBE request during which any message published to that topic may not yet be routed to that client, and the mqtt.js library used by both the CLI tool and my scripts does not surface this transient state clearly by default. This is a small but genuine illustration of the reliability trade-offs inherent to shared, best-effort infrastructure, echoing the unit's discussion of centralisation and synchronous coordination as challenges to building dependable scalable systems [6]. In a production deployment, this risk is mitigated by using Quality of Service (QoS) levels above zero, which the MQTT specification defines to guarantee at-least-once or exactly-once message delivery rather than the default best-effort delivery I used throughout this task [4].

Comparative Analysis: Command Line MQTT versus Node.js Scripts

Aspect	Command Line (mqtt CLI)	Node.js (client.js / server.js)
Purpose	Quick, ad hoc testing of topic behaviour and broker connectivity	Building a reusable, embeddable application component
Persistence	Process exists only for the duration of one command or session	Long-running process suitable for continuous drone telemetry
Extensibility	Limited to a single publish or subscribe action per invocation	Can incorporate application logic, e.g. reading sensor data, before publishing
Session and identity handling	Each invocation opens and closes its own short-lived connection to the broker, with no persistent client object to reference	Maintains one persistent client object (client) throughout the script's lifetime, allowing repeated publish or subscribe calls on the same connection
Error visibility	Failed or blocked commands, such as the PowerShell execution policy error encountered in this task, are not always surfaced clearly and may fail silently	Connection and delivery events are exposed explicitly through named callbacks (on('connect'), on('message')), giving clearer console feedback

Aspect	Command Line (mqtt CLI)	Node.js (client.js / server.js)
Message reliability (QoS)	Defaults to QoS 0 (best effort) unless a QoS flag is explicitly supplied	Also defaults to QoS 0 in the code used here, since both rely on the same underlying mqtt.js library; reliability is therefore identical unless QoS is configured explicitly
Best fit in this task	Verifying wildcard versus specific-topic filtering behaviour	Simulating the actual drone (publisher) and owner dashboard (subscriber) roles

Both approaches connect to the same underlying broker using the same MQTT protocol, and this exercise confirmed that behaviour observed at the command line, message delivery scoped to topic hierarchy and wildcard matching, is identical when reproduced through the Node.js client library, since both are simply different client implementations of the same open protocol specification [4]. The additional rows above, particularly session handling and error visibility, were drawn directly from the two troubleshooting episodes documented earlier in this report, rather than from general knowledge, and this is why the command line tool proved sufficient for quick verification while the Node.js scripts were the more appropriate choice for representing a persistent, long-running device role such as a drone or a monitoring dashboard.

Relevance to the Rent-a-Drone Scenario

Mapping this task's evidence back onto the fictional Rent-a-Drone scenario that motivates it, each drone in the fleet would run logic equivalent to my client.js script, continuously publishing its own telemetry, such as location, battery level, or altitude, to a topic scoped under its own unique drone identifier, for example /rentadrone/drone042/battery. The owner's central monitoring dashboard would run logic equivalent to my server.js script, but subscribed with a multi-level wildcard across the entire fleet namespace, for example /rentadrone/#, exactly mirroring the behaviour I demonstrated with the /jahan226100103/# subscription in Part 1. A field technician responsible for a single specific drone could instead subscribe narrowly to that one drone's topic branch, receiving only the telemetry relevant to their task and avoiding the noise of the rest of the fleet, precisely as demonstrated by my /jahan226100103/device subscriber. This mapping illustrates why MQTT's publish/subscribe model, rather than a request/response model such as REST, is considered well suited to IoT fleets of many independently moving devices: new drones can be added to the fleet and begin publishing without any existing subscriber needing reconfiguration, which is a direct practical demonstration of the scalability principle introduced earlier in this unit under the discussion of loosely coupled, event-driven system design [6]. The session-handling distinction identified in the comparative table above also reinforces why the persistent Node.js subscriber, rather than a repeated command line invocation, is the realistic model for a dashboard that must remain continuously connected to receive telemetry for as long as any drone in the fleet remains airborne.

Extra Effort: Supplementary Learning Resource

In addition to the unit's own lecture material on MQTT (Section 2.4, Lecture 8.2.1), I revisited the concept independently by watching a supplementary tutorial video, "What is MQTT Protocol? How it works? | 2022", published by IT and Automation Academy [7]. This video reinforced the broker-centric publish and subscribe model and the role of topics in filtering messages to interested clients, and its explanation of the broker as the component responsible for receiving, filtering, and redistributing messages closely matched both the unit's own description of MQTT and my own practical observations while completing this task, giving me additional confidence that my understanding of the underlying

mechanism was accurate before I began the hands-on exercises.

Reflection

This task gave me a considerably clearer, evidence-based understanding of why MQTT is described in the unit as suited to resource-constrained, high-scale IoT deployments, rather than simply accepting that description on faith from the lecture slides. Seeing the wildcard subscriber receive every published message while the specific-topic subscriber received only a subset, using the same broker and the same publisher, made the abstract idea of hierarchical topic filtering concrete in a way that reading about it alone would not have achieved. Working through the two genuine technical obstacles I encountered, the PowerShell execution policy restriction and the transient broker connectivity delay, was also valuable, since both are the kind of practical friction that arises in real deployments rather than idealised textbook examples, and diagnosing each one required distinguishing between a local environment configuration issue and a distributed systems reliability issue, which is itself a useful skill for the later modules of this unit. Building the comparative table also pushed me to think more carefully about why two tools implementing the identical protocol could still be suited to different purposes within the same task, rather than treating the command line and Node.js approaches as interchangeable. Building on my existing familiarity with the Arduino Nano 33 IoT and Arduino UNO R3 from earlier project work, I can now see clearly how those boards, most of which support the mqtt protocol through their networking libraries, could be substituted directly in place of my Node.js scripts to publish genuine sensor readings from a physical device rather than a hardcoded string message, and this is a direction I intend to explore further when working on this unit's Distinction project later in the trimester.

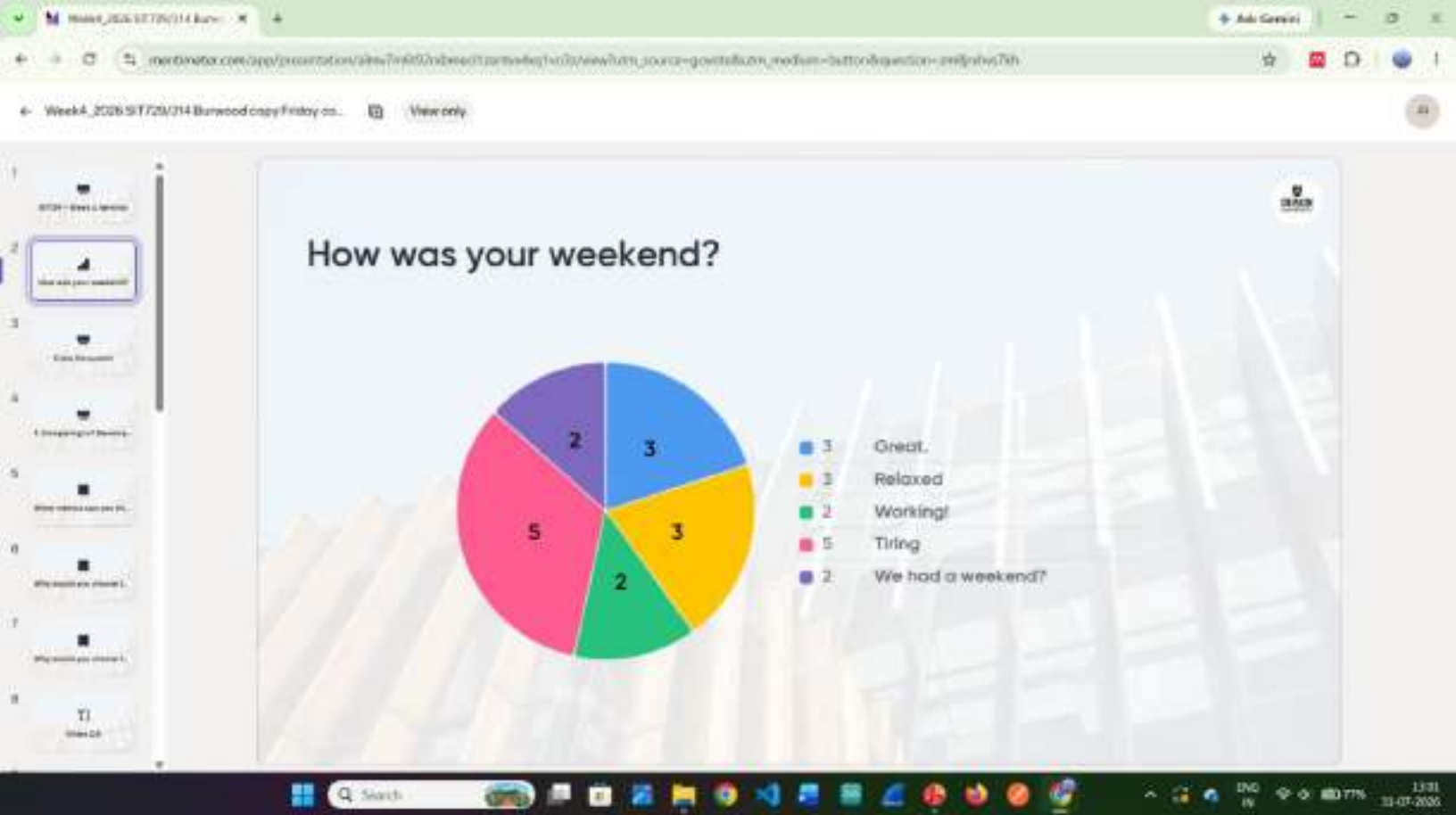
Key Notes

A small number of points are worth restating concisely for clarity when this task is reviewed. The mqtt multi-level wildcard character (#) must always appear as the final character of a topic filter and matches every level beneath the given prefix, whereas the single-level wildcard (+), which was not required for this particular task but is documented in the same unit lecture material, matches exactly one topic level and can be used in the middle of a topic filter [2][4]. Wildcards may only be used when subscribing to topics; a client can never publish a message to a topic that itself contains a wildcard character [4]. Finally, the unique topic prefix I adopted (/jahan226100103) is not a registered identifier of any kind, simply a self-chosen namespace convention to avoid colliding with other users of the same shared public broker, consistent with the cooperative broker-sharing guidance given in the unit material [2].

References

- [1] HiveMQ, "The free public MQTT broker by HiveMQ," HiveMQ, 2023. [Online]. Available: <https://www.hivemq.com/mqtt/public-mqtt-broker/>
- [2] J. Kua, "2.4 Message Queuing Telemetry Transport," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 2, Deakin University, Trimester 2, 2026.
- [3] OpenJS Foundation, "mqtt – npm," npm Inc., 2024. [Online]. Available: <https://www.npmjs.com/package/mqtt>
- [4] HiveMQ, "MQTT topics, wildcards, and best practices – MQTT Essentials: Part 5," HiveMQ Blog, 2019. [Online]. Available: <https://www.hivemq.com/blog/mqtt-essentials-part-5-mqtt-topics-best-practices/>
- [5] Microsoft, "About Execution Policies," Microsoft Learn, PowerShell documentation, 2024. [Online]. Available: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_execution_policies
- [6] J. Kua, "1.3 A Software Engineering Approach for Scalable IoT," SIT314/SIT729 Software Architecture and Scalability for Internet-of-Things, Module 1, Deakin University, Trimester 2, 2026.
- [7] IT and Automation Academy, "What is MQTT Protocol? How it works? | 2022," YouTube, 9 Apr. 2022. [Online]. Available: <https://www.youtube.com/watch?v=NXyf7tVsi10>

Academic Integrity Statement: All command line and Node.js output presented in this report was produced by my own execution of the described commands and scripts. All external sources are cited above in IEEE format at the point of use. This submission represents my own independent work.



Week4_2026 SIT/25/314 Barwood copy Friday as...

View only

2024 - Week 4 Lecture

How wide your workspace?

Class Discussion

4. Designing IoT Systems

What internet can you do?

Why should you choose L

Why should you choose L

Week 05

Class Discussion

1. Comparing IoT Development Platforms.

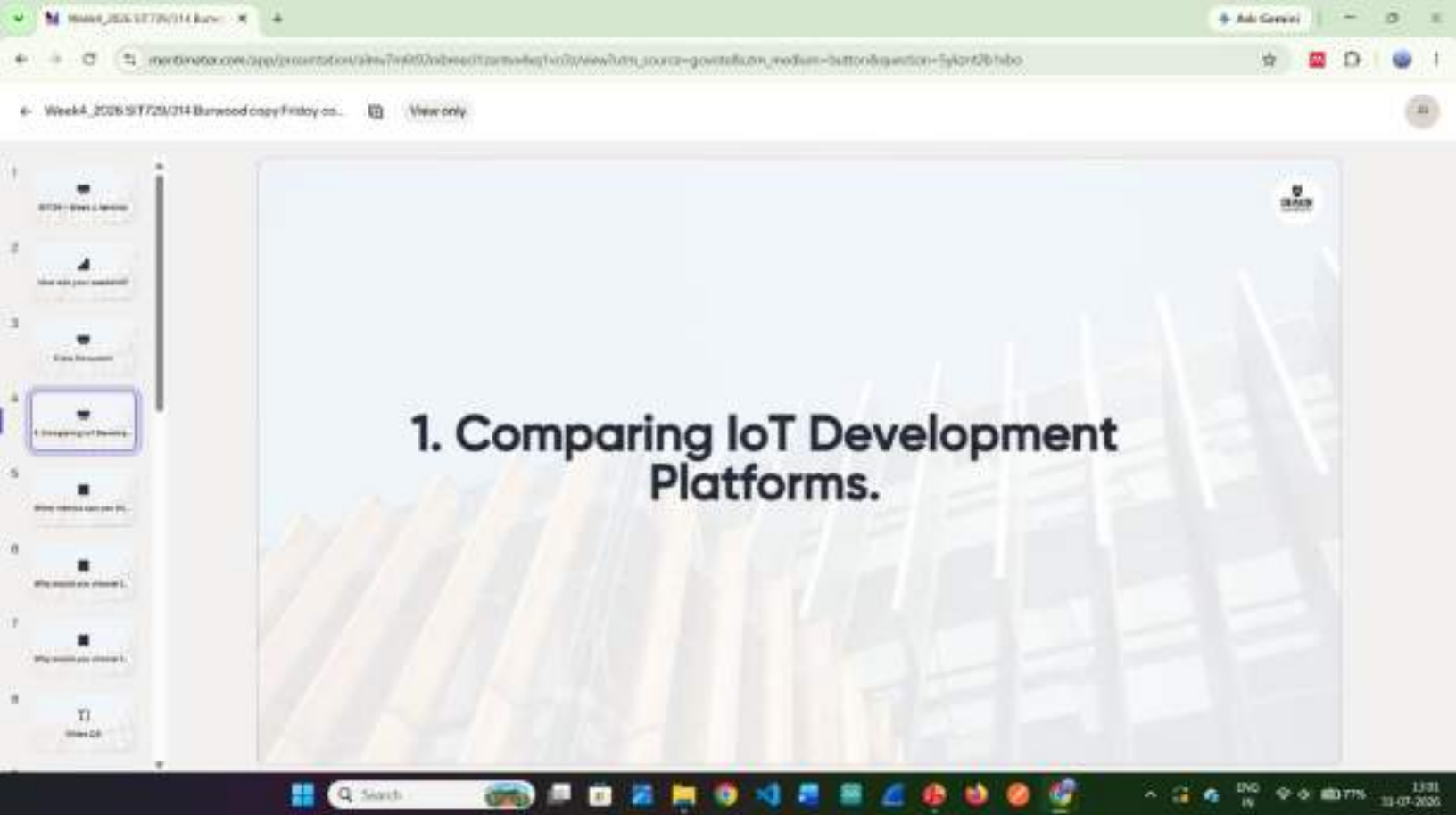
2. Choosing IoT Development Platforms.

3. Scaling IoT Development Platforms.

Search

13:31

31-07-2026



1. Comparing IoT Development Platforms.

2024 - Week 4 - Barwood

View only

What metrics can you think of to compare IoT development platforms?

bandwidth

scalability

reliability

regional availability

compute cost

functionality

bandwidth

user friendly

usability

capabilities

storage use

storage

ease of use

quality

security posture

popularity

distance

cost

performance

secure communication

security and reliability

sustainability

security

architecture

efficiency

Week4_2026 SIT/25/314 Barwood copy Friday as... View only

Why would you choose to use a full-service platform, e.g. Amazon or Microsoft IoT?

A word cloud of reasons for choosing a full-service platform. The words are arranged in a circular pattern around the center. The most prominent words are 'usability' (large blue), 'scalability' (large orange), and 'user friendly' (large pink). Other words include 'simplicity' (green), 'reliability' (green), 'high performance' (green), 'preconfigured' (green), 'depends on tech needed' (green), 'quality' (pink), 'time' (green), 'centralized' (green), 'support' (green), 'safe' (green), 'massive scale' (green), 'faster deployment' (green), 'deployment support' (green), 'high user count' (green), 'secure' (green), 'cost' (blue), 'reliable' (pink), 'design ecosystem integration' (yellow), and 'cost-effective' (yellow).

usability

scalability

user friendly

simplicity

reliability

high performance

preconfigured

depends on tech needed

quality

time

centralized

support

safe

massive scale

faster deployment

deployment support

high user count

secure

cost

reliable

design ecosystem integration

cost-effective

Week4_2026 SIT/25/314 Barwood copy Friday as...

View only

2024 - Week 4 content

How wide your window?

Extra Resources

4. Designing your Network

What network can you do?

Why would you choose L...

Why would you choose L...

What OS

Why would you choose to use an open source approach, e.g. Node-Red, ESPHome, Zigbee2MQTT?

control over own data

extensive documentation

reliability

flexibility

community support

simulation

lower cost

zero tracking

trustworthy

scalable

allow full customisation

trustworthiness

digital sovereignty

preconfigured

future developments

the principle of free

customisable

control

principle

cheap

cost

Search

77%

13:31 31-07-2026

2024 - 2025 Academic

How will your workload?

Classroom

4. Designing your Learning

What resources can you use?

Why should you attend?

Why should you attend?

11 Slides 2.8

Slides 2.8

2. Choosing IoT Development Platforms

- Samsung SmartThings
- Apple HomeKit
- Home Assistant
- Google Home
- Fiware
- Node-RED
- AWS IoT Core
- Azure IoT Central
- ThingsBoard
- IFTTT
- ESPHome
- Zigbee2MQTT
- Tuya
- others...

Week4_2026 SIT/25/314 Barwood copy Friday as... View only

What IoT platform would you use for building a smart home?

esphome node-red
google home
apple homekit
home assistant iftt
samsung smartthings

applehomekit I wouldn't smartthings

13:31 33-07-2026

What IoT platform would you use for building the new Melbourne public transport contactless payment system?

A word cloud featuring various cloud services and IoT-related terms. The most prominent word is 'aws' in large blue letters. Other visible words include 'azure' in large yellow letters, 'international contractor' in red, 'zigbee' in orange, 'microsoft services' in grey, 'azure lot central' in green, 'atlas ops' in grey, and 'azure iot' in green. The background is a light grey grid.

Week4_2026 SIT/25/314 Barwood copy Friday as... View only

What IoT platform would you use for building a smart watch health monitoring system?

aws

apple healthkit
aws and nodered
thingsboard
cloud
azure
thingboard
aws lot core
healthcare api
things board
azure iot central
node-red
google cloud
mqtt
apple homekit
samsung smarthings

13:31 31-07-2020

Week4_2026 SIT/25/314 Barwood copy Friday as... View only

For a smart home, if you increased the number of homes, what would you need to scale?

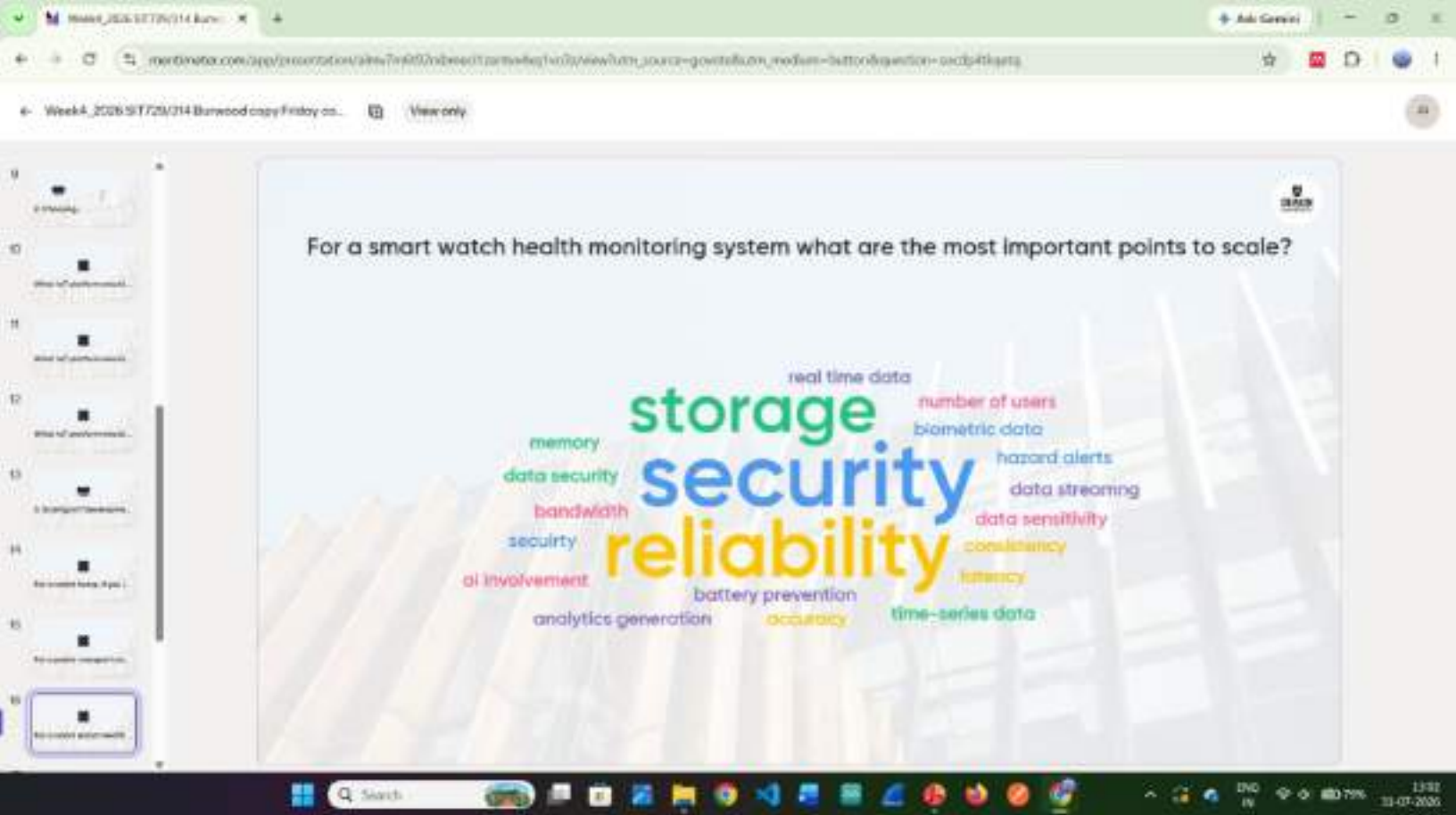
A word cloud on a background of a modern building facade. The words are of various sizes and colors, representing factors that need to be scaled for a smart home system. The most prominent words are 'data storage' (large blue), 'bandwidth' (large orange), 'sensors' (large red), and 'servers' (large purple). Other visible words include 'performance', 'load balancing', 'data pipeline', 'broker', 'sensor range', 'sensor count', 'no of users', 'number of switches', 'cloud infrastructure', 'number of devices', 'routers', 'no of sensors', and 'backend'.

performance load balancing data pipeline
no of sensors backend broker
data storage sensor range
no of users bandwidth sensor count
number of switches servers
cloud infrastructure sensors routers
number of devices



For a public transport contactless payment system what are the most important points to scale?





For a smart watch health monitoring system what are the most important points to scale?

storage
security
reliability
real time data
number of users
biometric data
hazard alerts
data streaming
data sensitivity
consistency
latency
time-series data
accuracy
battery prevention
analytics generation
ai involvement
security
bandwidth
data security
memory
hazard alerts



Jahan Garg | Student ID: 226100103

SIT314/SIT729 – Week 4 Group Activity

Designing Smart IoT Solutions

Overview

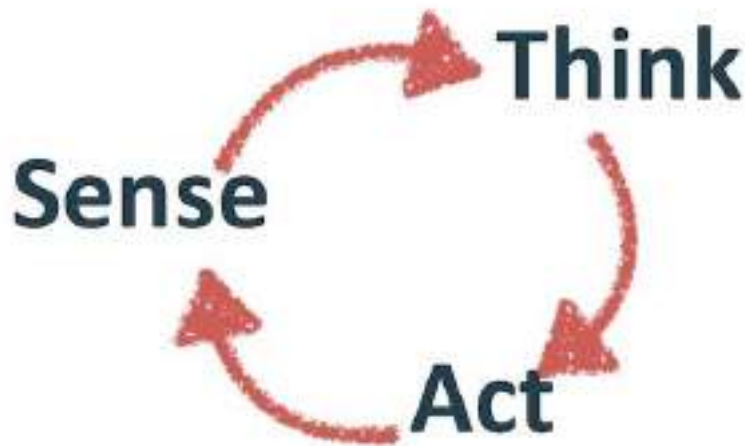
IoT applications typically comprise several interdependent stages: data collection with sensors, data aggregation, filtering, storage, analysis, and eventual actuation in the physical environment, such as switching a traffic light phase or raising a level crossing barrier (Module 2.8, SIT314/SIT729). This activity applies that pipeline to a substantially larger and more complex scenario than the single vehicle case studied in the Week 3 Group Activity: a transport system for a smart city, encompassing private vehicles, public transport, pedestrians, and the traffic management infrastructure that coordinates all of them. The design presented below is structured around the Sense, Think, Act loop introduced in the unit's Smart Platforms material, in which sensing components collect data from the managed environment, thinking components analyse that data and update an internal model of the environment, and actuation components put decisions into effect and change the state of the world (Lee, Lecture 2.8, SIT314/SIT729).

This is an individually authored submission. The scenario and initial framing were discussed collaboratively with classmates Melissa Woo and Madhur Gupta during the Week 4 seminar; however, all analysis, architectural decisions, diagrams, and written content presented below are my own independent work.

Tasks

You should design the IoT flow for a 'transport system for a smart city'.

When thinking about this, think about Sense-Think-Act.



1. Sketch a block diagram of the system.

1. Block Diagram of the System

The block diagram in Figure 1 organises the smart city transport system into three tiers, consistent with the N-tier Edge, Fog, Cloud deployment architecture introduced in Module 1.3 (Kua, Lecture 1.3, SIT314/SIT729) and applied previously to the Week 3 Vehicle Asset Health scenario. The edge tier consists of in-vehicle units, intersection controllers, and roadside sensors performing local sensing and immediate, safety-critical actuation. The fog tier consists of precinct-level traffic management nodes that coordinate a cluster of intersections or a transport corridor, allowing faster-than-cloud responses across a local area. The cloud tier consists of a single, city-wide Traffic Control Centre platform responsible for city-scale analytics, congestion prediction, and public transport dispatch optimisation. Solid arrows in the diagram represent sensing data flowing upward through the tiers, while dashed arrows represent actuation commands flowing back down to the physical environment.

Figure 1. Block Diagram of the Smart City Transport System (Sense-Think-Act across Edge, Fog and Cloud tiers)

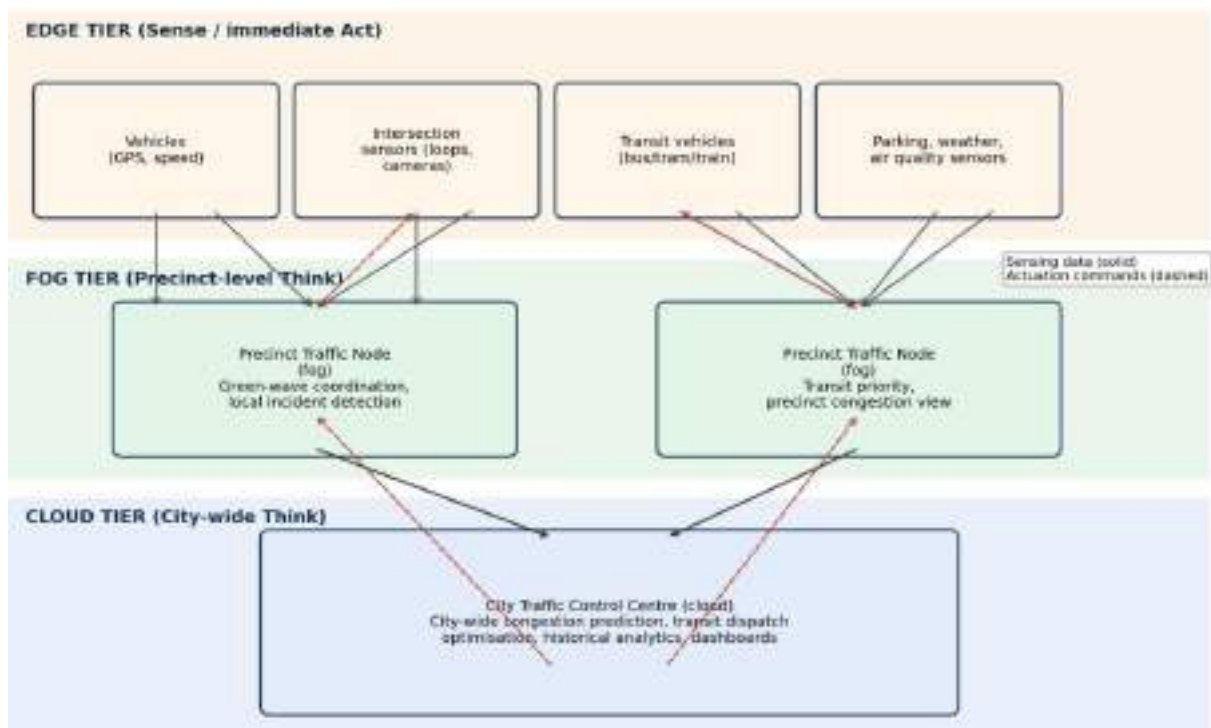


Figure 1. Block diagram of the Smart City Transport System, showing the Sense, Think, Act loop mapped across the Edge, Fog and Cloud tiers.

2. What are all the physical objects in the space (cars, people, traffic lights)?

2. Physical Objects in the Space

The scenario involves a heterogeneous mix of people, places and things. These are summarised in Table 1, following the same people, places and things framing applied in the Week 3 Group Activity (Kua, Lecture 1.3, SIT314/SIT729), here scaled up from a single mine site to an entire city transport network.

Category	Physical Objects
People	Drivers, pedestrians, cyclists, public transport passengers, traffic control operators, emergency service personnel
Vehicles	Private cars, buses, trams, trains, taxis and rideshare vehicles, emergency vehicles (ambulance, fire, police), delivery vehicles
Road infrastructure	Traffic lights, pedestrian crossing signals, variable message signs, roadside inductive loop sensors, CCTV cameras, weather stations, EV charging stations, parking sensors
Rail and transit infrastructure	Train and tram platforms, level crossings and boom gates, transit signal priority beacons, ticketing gates
Places	Intersections, arterial roads, precinct traffic zones, transit stations, congestion pricing zones, car parks

Control systems	Precinct-level traffic management nodes (fog), the city-wide Traffic Control Centre (cloud)
------------------------	---

3. What is the data that is generated from sensors (car locations, traffic)?

3. Data Generated from Sensors

Following the sensing stage of the Sense, Think, Act loop (Lee, Lecture 2.8, SIT314/SIT729), the system generates a wide variety of sensor data across the tiers identified in Figure 1. Table 2 summarises the principal data sources and the readings each one produces.

Source	Sensor Data Generated
Vehicles (private and public)	GPS location, speed, heading, occupancy or passenger count (public transport), fuel or battery level
Intersection sensors	Inductive loop vehicle presence and counts, pedestrian call button presses, current signal phase and timer state
Traffic cameras	Vehicle counts, average speed, queue length, incident or collision detection via computer vision
Parking sensors	Bay occupancy status, EV charger availability
Weather stations	Rainfall, visibility, road surface temperature, relevant to speed limit and signal timing decisions
Rail and tram sensors	Train and tram position, speed, and approach distance to level crossings
Air quality sensors	Particulate matter and emissions levels near arterial roads
Ticketing and access systems	Passenger boarding counts, congestion zone entry events

As discussed in the Week 1 Group Activity task of quantifying data volume for IoT applications, a city-scale transport deployment involves a very high number of concurrently reporting nodes, each transmitting small, frequent payloads. This profile, as covered in Module 2.3, is not well suited to LPWAN technologies alone, since LPWAN is characterised as suited to delay-tolerant use cases exchanging small amounts of data infrequently rather than continuous, low-latency traffic streams. A realistic deployment therefore requires a mix of connectivity technologies depending on each sensor's location, mobility, and latency requirements (Kua, Lecture 3.3, SIT314/SIT729).

4. What are the actuations of the system (traffic lights, car navigation, train speeds)?

4. Actuations of the System

The actuation stage closes the Sense, Think, Act loop by putting decisions into effect in the physical environment (Lee, Lecture 2.8, SIT314/SIT729). Table 3 summarises the principal actuations available in this design.

Actuation	Purpose
Traffic light phase change	Adjust green and red timing in response to real-time traffic flow
Variable speed limit signs	Reduce speed limits dynamically in poor weather or near an incident
Dynamic lane management signage	Open or close lanes, such as tidal flow lanes, based on directional demand
Emergency vehicle signal pre-emption	Force a green phase for an approaching emergency vehicle
Public transport signal priority	Extend a green phase for an approaching bus or tram to reduce delay
Driver navigation rerouting alerts	Push live rerouting suggestions to connected vehicle navigation systems
Pedestrian crossing signal control	Extend crossing time for a detected mobility-impaired pedestrian
Rail level crossing boom gates	Lower or raise gates in response to an approaching train
Parking guidance signage	Direct drivers to the nearest available parking bay or EV charger
Public transport dispatch adjustment	Add services or reroute buses and trams in response to detected demand or delay

5. What are the processes and decisions that need to be made to connect the sensors to the actuators?

5. Processes and Decisions Connecting Sensors to Actuators

This is the Think stage of the loop. Consistent with the edge, fog, cloud processing split established in the Week 3 Group Activity (Kua, Lecture 1.3, SIT314/SIT729), decision making in this design is distributed according to the urgency and geographic scope of each decision, as summarised in Table 4.

Tier	Processing and Decisions Performed
Edge (intersection controller)	Immediate local signal timing adjustment based on inductive loop counts at that intersection; instant emergency vehicle pre-emption when a priority beacon is detected. These decisions cannot wait for a cloud round trip given the safety-critical, sub-second latency required.
Fog (precinct traffic node)	Coordinates signal timing across a cluster of adjacent intersections, for example a green wave progression along an arterial road; aggregates precinct-level congestion data; detects a developing incident from correlated camera and

	loop data before escalating it to the cloud.
Cloud (Traffic Control Centre)	City-wide congestion prediction and modelling; public transport network-level dispatch optimisation; historical trend analysis for infrastructure planning; cross-precinct coordination during major events or incidents; generation of city-wide dashboards for operators.

This distribution reflects the same rationale applied in Week 3: continuously streaming every raw sensor reading from an entire city to a single cloud platform would be both bandwidth prohibitive and too slow for safety-critical actuation, such as emergency vehicle pre-emption. The majority of time-critical decisions are therefore resolved as close to the sensor as possible, with only aggregated or exceptional data escalated upward to the fog and cloud tiers (Kua, Lecture 1.3, SIT314/SIT729).

6. Plan a hierarchical MQTT application-level communications protocol. Think about the identity of users and devices in the system and plan single-level or multi-level wildcards for message filtering.

6. Hierarchical MQTT Application-Level Communications Protocol

As covered in Module 2.4, MQTT topics are hierarchical strings separated by a forward slash, and clients may subscribe using single-level (plus sign) or multi-level (hash sign) wildcards for message filtering (Kua, Lecture 8.2.1, SIT314/SIT729). Because a city may contain multiple precincts, each with many intersections and transit routes, the topic hierarchy below is rooted at the city level, descending through precinct, then asset type, then asset identifier. This follows the same cooperative broker sharing convention discussed in the unit, of prefixing topics with a unique identifier to avoid collisions on a shared broker (Kua, Lecture 8.2.1, SIT314/SIT729).

6.1 Topic Hierarchy

Topic	Purpose
city/{precinctID}/intersection/{intersectionID}/sensor/loop/count	Vehicle count from an inductive loop sensor
city/{precinctID}/intersection/{intersectionID}/signal/phase	Current traffic light phase or state
city/{precinctID}/intersection/{intersectionID}/camera/incident	Incident or collision detection alert
city/{precinctID}/transit/{vehicleID}/location	GPS location of a bus, tram or train

city/{precinctID}/transit/{vehicleID}/occupancy	Passenger occupancy level
city/{precinctID}/parking/{lotID}/availability	Parking bay or EV charger availability
city/{precinctID}/weather/station/{stationID}	Local weather and road surface readings
city/{precinctID}/emergency/{vehicleID}/preemption	Emergency vehicle signal pre-emption request
city/{precinctID}/crossing/{crossingID}/gate/status	Rail level crossing boom gate status
city/{precinctID}/command/signal/{intersectionID}	Cloud or fog-to-intersection actuation command

6.2 Identity Design

Each intersection controller, transit vehicle, and sensor authenticates to the broker with a unique client identifier and publishes only under its own branch of the hierarchy. For example, a given intersection controller may publish only to city/{precinctID}/intersection/{itsOwnID}/, with write access restricted by a broker Access Control List, mirroring the identity design used in the Week 3 Group Activity. Human roles, including the precinct traffic operator, the city-wide Traffic Control Centre analyst, and the emergency dispatch operator, authenticate with role-specific credentials and are granted subscribe-only access to the branches of the hierarchy that are relevant to their responsibilities.

6.3 Wildcard Subscription Design

Stakeholder	Subscription Topic Filter	Rationale
Single intersection controller	city/precinct04/intersection/INT-012/#	Multi-level wildcard. Receives every sensor and command topic for its own intersection only.
Precinct traffic node (fog)-	city/precinct04/intersection/+/#	Single-level wildcard on intersection identifier, multi-level beneath it. Aggregates every intersection in the precinct without needing to know how many intersections exist.
City Traffic Control Centre (cloud)	city/+/transit/+/location	Single-level wildcards on both precinct and vehicle identifier.

		Tracks every transit vehicle city-wide for network-level dispatch decisions.
Emergency dispatch operator	city/+/emergency/+/preemption	Cross-precinct visibility of every emergency pre-emption event, regardless of which precinct or vehicle raised it.
Maintenance engineer (rail crossings only)	city/+/crossing/#	Fleet-wide visibility of all level crossing gate status, ignoring unrelated traffic and transit topics.

This design directly demonstrates the distinction between the two wildcard types taught in Module 2.4: the plus sign substitutes exactly one topic level, such as any precinctID or intersectionID, while the hash sign subscribes to an entire branch and everything beneath it regardless of depth (Kua, Lecture 8.2.1, SIT314/SIT729). Structuring the hierarchy this way means that as the city adds new precincts, intersections, or transit vehicles, no existing subscriber needs to be reconfigured. New publishers simply begin publishing under the existing hierarchy and are automatically picked up by any wildcard subscription that already covers that branch, which is the same scalability property demonstrated with the /jahan226100103/# subscription in the Week 3 Technical Task.

Figure 2. Hierarchical MQTT Topic Tree for the Smart City Transport System

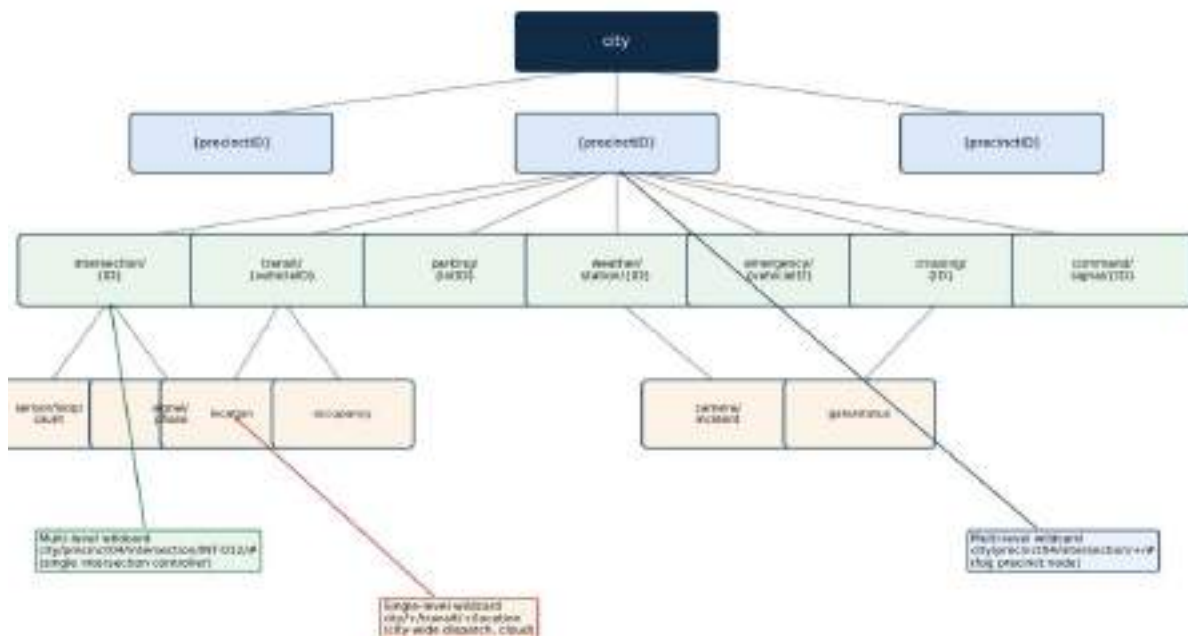


Figure 2. Hierarchical MQTT topic tree for the Smart City Transport System, showing the city, precinct, asset type, and asset identifier branching structure, together with the wildcard subscriptions used by three representative stakeholders.

7. Reflection

Designing the IoT flow for a smart city transport system reinforced how quickly complexity grows once a scenario moves from a single asset, as in the Week 3 Vehicle Asset Health activity, to a city-wide network of heterogeneous, interacting devices. The Sense, Think, Act framing was particularly useful for keeping the design grounded. Rather than attempting to design the whole system at once, decomposing it into what is sensed, what must be decided, and what is actuated made it much easier to reason about where each decision should physically be made, a judgement that, as in Week 3, depends on latency and safety criticality rather than convenience. Planning the MQTT hierarchy also highlighted a subtlety that was less visible in the single vehicle scenario. Because a city contains many independent precincts and asset types operating concurrently, the topic hierarchy has to be designed so that fog level aggregation, per precinct, and cloud level aggregation, city-wide, can both be expressed through wildcards without requiring two separate publishing schemes. This is a pattern I intend to carry into my Distinction project, particularly when deciding how granular a topic hierarchy needs to be before it becomes unwieldy to design and maintain.

References

Kua, J. Lecture 1.3 - A Software Engineering Approach for Scalable IoT. SIT314/SIT729, Deakin University, Trimester 2, 2026.

Kua, J. Lecture 3.3 - Low-Power Wide-Area Networks. SIT314/SIT729, Deakin University, Trimester 2, 2026.

Kua, J. Lecture 8.2.1 - Message Queuing Telemetry Transport. SIT314/SIT729, Deakin University, Trimester 2, 2026.

Lee, K. Lecture 2.8 - Smart Platforms. SIT314/SIT729, Deakin University, Trimester 2, 2026.

Academic Integrity Statement: This is an individually authored submission. The scenario and initial framing were discussed collaboratively with classmates Melissa Woo and Madhur Gupta during the Week 4 seminar; however, all analysis, architectural decisions, diagrams, and written content presented above are my own independent work.